Reindexing

Reindexing changes the row labels and column labels of a DataFrame. To reindex means to conform the data to match a given set of labels along a particular axis.

Multiple operations can be accomplished through indexing like −Reorder the existing data to match a new set of labels. Insert missing value (NA) markers in label locations where no data for the label existed.

```
import pandas as pd
import numpy as np
N=20
df = pd.DataFrame({
#synthetic data generation ie generating data
  'A': pd.date_range(start='2016-01-01',periods=N,freq='D'),\
  'x': np.linspace(0,stop=N-1,num=N),\
  'y': np.random.rand(N),\
  'C': np.random.choice(['Low','Medium','High'],N),\
  'D': np.random.normal(100, 10, size=(N))\
})
print(df)
#reindex the DataFrame
df_reindexed = df.reindex(index=[0,2,5], columns=['A', 'C',\
'B'])
print(df_reindexed)
```

Output:

	A	x	y	C	D
0	2016-01-01	0.0	0.630052	High	97.538064
1	2016-01-02	1.0	0.676587	Medium	110.840490
2	2016-01-03	2.0	0.912597	Low	123.289745
3	2016-01-04	3.0	0.057429	High	110.777584
4	2016-01-05	4.0	0.759413	High	93.724167
5	2016-01-06	5.0	0.782925	High	98.244549
6	2016-01-07	6.0	0.987753	Low	94.739458
7	2016-01-08	7.0	0.679728	High	89.823543
8	2016-01-09	8.0	0.190942	High	117.974507
9	2016-01-10	9.0	0.136869	Medium	122.884714
10	2016-01-11	10.0	0.421413	Medium	100.896791
11	2016-01-12	11.0	0.684511	Medium	73.513576
12	2016-01-13	12.0	0.194240	Low	91.648559
13	2016-01-14	13.0	0.849464	Low	101.412134
14	2016-01-15	14.0	0.612285	Low	103.659118
15	2016-01-16	15.0	0.251048	High	108.285121
16	2016-01-17	16.0	0.738210	Low	102.542300
17	2016-01-18	17.0	0.290101	High	107.300025

```
18 2016-01-19  18.0  0.556455    High  102.461087

19 2016-01-20  19.0  0.766565  Medium   99.856390

           A     C   B

0 2016-01-01  High NaN

2 2016-01-03   Low NaN

5 2016-01-06  High NaN
```

Iteration of columns and rows

Iterating a DataFrame. Iterating a DataFrame gives column names

```
import pandas as pd
import numpy as np
N = 20
df = pd.DataFrame({\
    'A': pd.date_range(start='2016-01-01', periods=N,\
         freq='D'),\
    'x': np.linspace(0, stop=N - 1, num=N),\
    'y': np.random.rand(N),\
    'C': np.random.choice(['Low', 'Medium', 'High'],\ N),\
    'D': np.random.normal(100, 10, size=(N))\
})
print(df)

for col in df:
    print(col)
```

Output:

	A	x	y	C	D	
0	2016-01-01	0.0	0.493200	Low	109.160764
1	2016-01-02	1.0	0.952512	High	102.391040
2	2016-01-03	2.0	0.273505	Low	83.758507
3	2016-01-04	3.0	0.552651	High	108.316930
4	2016-01-05	4.0	0.596099	High	79.882069
5	2016-01-06	5.0	0.646804	Low	110.857690
6	2016-01-07	6.0	0.853289	Medium	115.655619
7	2016-01-08	7.0	0.771558	Low	97.186514
8	2016-01-09	8.0	0.825620	Low	99.327691
9	2016-01-10	9.0	0.621610	High	100.459352
10	2016-01-11	10.0	0.821983	High	90.170310
11	2016-01-12	11.0	0.151487	Medium	103.658817
12	2016-01-13	12.0	0.358558	High	118.000452
13	2016-01-14	13.0	0.249145	High	80.884518
14	2016-01-15	14.0	0.443094	Medium	100.578761
15	2016-01-16	15.0	0.164142	Low	106.104461
16	2016-01-17	16.0	0.520943	Low	101.728910
17	2016-01-18	17.0	0.979398	Medium	95.033657

```
18 2016-01-19  18.0  0.245693  Medium  100.520263
19 2016-01-20  19.0  0.284320  Medium   92.331131
A
x
y
C
D
```

To iterate over the rows of the DataFrame, we can use the following functions

iterrows() − iterate over the rows as (index,series) pairs

itertuples() − iterate over the rows as named tuples

1.iterrows()

iterrows() returns the iterator yielding each index value along with a series containing the data in each row.

```
import pandas as pd
import numpy as np
df = pd.DataFrame(np.random.randn(4,3),columns =\
     ['col1','col2','col3'])
print(df)
for row_index,row in df.iterrows():
   print("Row Index:",row_index)
   print("Row")
   print(row)
```

```
Output:
col1      col2      col3
0 -1.067551 -1.367131  0.784553
1 -1.076551  1.178573 -2.011397
2  0.326878 -0.726365 -0.975675
3 -0.145776 -1.021032 -0.005339
Row Index: 0
Row
col1   -1.067551
col2   -1.367131
col3    0.784553
Name: 0, dtype: float64
Row Index: 1
Row
col1   -1.076551
col2    1.178573
col3   -2.011397
Name: 1, dtype: float64
Row Index: 2
Row
```

```
col1    0.326878

col2   -0.726365

col3   -0.975675

Name: 2, dtype: float64

Row Index: 3

Row

col1   -0.145776

col2   -1.021032

col3   -0.005339

Name: 3, dtype: float64
```

2.itertuples()

itertuples() method will return an iterator yielding a named tuple for each row in the DataFrame. The first element of the tuple will be the row's corresponding index value, while the remaining values are the row values

```
import pandas as pd
import numpy as np
df = pd.DataFrame(np.random.randn(4,3),columns =\
     ['col1','col2','col3'])
print(df)
for row in df.itertuples():
    print(row)
```

Output:

```
col1      col2      col3

0  0.570976 -0.732205  0.547647

1 -0.527166 -0.448373 -1.358177

2 -1.478145 -1.210618  0.345836

3 -1.389563  0.893142 -0.425654

Pandas(Index=0,        col1=0.5709756614296673,        col2=-
0.7322054649018599, col3=0.5476465403712114)

Pandas(Index=1,        col1=-0.5271662012068751,        col2=-
0.44837302624320974, col3=-1.3581766395481367)

Pandas(Index=2,        col1=-1.4781454539433028,        col2=-
1.2106180498058468, col3=0.3458358483364195)

Pandas(Index=3,                    col1=-1.3895631785124278,
col2=0.8931420073053614, col3=-0.4256542428323461)
```

Sorting

There are two kinds of sorting available in Pandas. They are –

- By label
- By Actual Value

```
import pandas as pd
import numpy as np
unsorted_df=pd.DataFrame(np.random.randn(10,2),\
            index=[1,4,6,2,3,5,9,8,0,7],\
```

```
            columns=['col2','col1'])
print (unsorted_df)
```

Output:

```
col2      col1
1 -1.816130 -1.921126
4 -0.457046 -0.410964
6  0.747339 -0.778212
2 -1.531689  0.457692
3  0.655275  0.760035
5  0.234617 -0.030996
9 -0.695533 -0.900700
8 -0.022268  0.872922
0  0.968679  0.083168
7  0.577277  1.728254
```

By Label

Using the sort_index() method, by passing the axis arguments and the order of sorting, DataFrame can be sorted. By default, sorting is done on row labels in ascending order.

```
import pandas as pd
import numpy as np
unsorted_df= pd.DataFrame(np.random.randn(10,2),\
             index=[1,4,6,2,3,5,9,8,0,7],\
```

```
            columns = ['col2','col1'])
print(unsorted_df)
sorted_df=unsorted_df.sort_index()
print(sorted_df)
```

Output:

```
col2      col1
1  1.185557 -0.985647
4  0.475840 -1.287246
6  1.319632  1.343245
2 -0.138574  0.709059
3  2.116719  0.888167
5 -0.166571 -1.074135
9  0.229410  0.259872
8 -0.343826  1.959523
0 -0.270768 -0.807570
7  0.850531  0.076390
       col2      col1
0 -0.270768 -0.807570
1  1.185557 -0.985647
2 -0.138574  0.709059
3  2.116719  0.888167
```

4 0.475840 -1.287246

5 -0.166571 -1.074135

6 1.319632 1.343245

7 0.850531 0.076390

8 -0.343826 1.959523

9 0.229410 0.259872

Order of Sorting

By passing the Boolean value to ascending parameter, the order of the sorting can be controlled. Let us consider the following example to understand the same.

```
import pandas as pd
import numpy as np
unsorted_df=pd.DataFrame(np.random.randn(10,2),
            index=[1,4,6,2,3,5,9,8,0,7],\
            columns = ['col2','col1'])
sorted_df = unsorted_df.sort_index(ascending=False)
print(sorted_df)
```

Output:

col2   col1

9 -0.242210 -1.336839

8 1.482145 0.558896

7 1.581501 0.318474

```
6 -0.057682 -2.184021

5  2.649003 -0.158563

4  0.590334  0.514185

3  0.734827 -1.147293

2 -0.388997 -0.385994

1  0.867097 -1.433076

0  1.752885 -0.696605
```

Sort the Columns

By passing the axis argument with a value 0 or 1, the sorting can be done on the column labels. By default, axis=0, sort by row.

```
import pandas as pd
import numpy as np
unsorted_df = pd.DataFrame(np.random.randn(10, 2),
              index=[1, 4, 6, 2, 3, 5, 9, 8, 0, 7],\
              columns = ['col2', 'col1'])
sorted_df = unsorted_df.sort_index(axis=1)
print(sorted_df)
```

Output:

```
col1      col2

1  1.071292  0.894443

4 -0.460073  0.872611

6  0.719253 -0.027978
```

```
2  0.555485 -0.127625

3 -1.603604  1.708652

5  0.852094 -0.358224

9  3.166687  0.320809

8 -0.804483 -0.652849

0  0.917012 -0.416454

7  1.545603 -0.529736
```

By Value

Like index sorting, sort_values() is the method for sorting by values. It accepts a 'by' argument which will use the column name of the DataFrame with which the values are to be sorted.

```
import pandas as pd
import numpy as np
unsorted_df=pd.DataFrame({'col1':[2,1,1,1],\
                          'col2':[1,3,2,4]})
print(unsorted_df)
sorted_df = unsorted_df.sort_values(by='col1')
print(sorted_df)
```

Output:

```
col1  col2

0     2     1

1     1     3
```

```
2     1     2
3     1     4
   col1  col2
1     1     3
2     1     2
3     1     4
0     2     1
```

'by' argument takes a list of column values.

```
import pandas as pd
import numpy as np
unsorted_df=pd.DataFrame({'col1':[2,1,1,1],\
                          'col2':[1,3,2,4]})
print(unsorted_df)
sorted_df = unsorted_df.sort_values(by=['col1','col2'])
print(sorted_df)
```

Output:

```
col1  col2
0     2     1
1     1     3
2     1     2
3     1     4
   col1  col2
```

2 1 2

1 1 3

3 1 4

0 2 1

Sorting Algorithm

sort_values() provides a provision to choose the algorithm from mergesort, heapsort and quicksort. **Mergesort is the only stable algorithm.**

```
import pandas as pd
import numpy as np
unsorted_df=pd.DataFrame({'col1':[2,1,1,1],\
                          'col2':[1,3,2,4]})
print(unsorted_df)
sorted_df=unsorted_df.sort_values(by='col1',kind='mergesort'
)
print(sorted_df)
```

Output:

col1 col2

0 2 1

1 1 3

2 1 2

3 1 4

col1 col2

```
1     1     3

2     1     2

3     1     4

0     2     1
```

Working with Text Data

The string operations with our basic Series/Index

1. lower()

```
import pandas as pd
import numpy as np
s = pd.Series(['Tom', 'William Rick', 'John',\
               'Alber@t', np.nan, '1234','SteveSmith'])
print(s.str.lower())
```

```
Output:

0             tom

1    william rick

2            john

3         alber@t

4             NaN

5            1234

6      stevesmith

dtype: object
```

2. upper()

```
import pandas as pd
import numpy as np
s = pd.Series(['Tom', 'William Rick', 'John',\
               'Alber@t', np.nan, '1234','SteveSmith'])
print (s.str.upper())
```

Output:

```
0              TOM
1    WILLIAM RICK
2             JOHN
3          ALBER@T
4              NaN
5             1234
6       STEVESMITH
dtype: object
```

3. len()

```
import pandas as pd
import numpy as np
s = pd.Series(['Tom', 'William Rick', 'John',\
               'Alber@t', np.nan, '1234','SteveSmith'])
```

```
print(s.str.len())
```

Output:

```
0     3.0
1    12.0
2     4.0
3     7.0
4     NaN
5     4.0
6    10.0
dtype: float64
```

4. strip()

```
import pandas as pd
import numpy as np
s = pd.Series(['Tom ', ' William Rick', 'John',\
               'Alber@t'])
print(s)
print ("After Stripping:")
print(s.str.strip())
```

Output:

```
0             Tom
1     William Rick
```

```
2              John
3           Alber@t
dtype: object
After Stripping:
0              Tom
1    William Rick
2              John
3           Alber@t
dtype: object
```

5. split()

```
import pandas as pd
import numpy as np
s = pd.Series(['Tom ', ' William Rick', 'John',\
               'Alber@t'])
print(s)
print ("Split Pattern:")
print(s.str.split(' '))
Output:
0              Tom
1     William Rick
2              John
3           Alber@t
```

```
dtype: object

Split Pattern:

0              [Tom, ]

1    [, William, Rick]

2               [John]

3            [Alber@t]

dtype: object
```

6. contains()

```
import pandas as pd
s = pd.Series(['Tom ', ' William Rick', 'John',\
               'Alber@t'])
print(s.str.contains(' '))
```

```
Output:

0     True

1     True

2    False

3    False

dtype: bool
```

7. replace()

```
import pandas as pd
```

```
s = pd.Series(['Tom ', ' William Rick', 'John',\
               'Alber@t'])
print(s)
print ("After replacing @ with $:")
print(s.str.replace('@','$'))
```

Output:

```
0             Tom
1     William Rick
2             John
3          Alber@t
dtype: object
After replacing @ with $:
0             Tom
1     William Rick
2             John
3          Alber$t
dtype: object
```

8. count()

```
import pandas as pd
s = pd.Series(['Tom ', ' William Rick', 'John',\
               'Alber@t'])
print("The number of 'm's in each string:")
```

```
print(s.str.count('m'))
```

Output:

The number of 'm's in each string:

0   1

1   1

2   0

3   0

dtype: int64

9. startswith()

```
import pandas as pd
s = pd.Series(['Tom ', ' William Rick', 'John',\
               'Alber@t'])
print ("Strings that start with 'T':")
print(s.str. startswith ('T'))
```

Output:

Strings that start with 'T':

0   True

1   False

2   False

3   False

dtype: bool

10. endswith()

```
import pandas as pd
s = pd.Series(['Tom ', ' William Rick', 'John',\
               'Alber@t'])
print ("Strings that end with 't':")
print(s.str.endswith('t'))
```

Output:

```
Strings that end with 't':
0    False
1    False
2    False
3     True
dtype: bool
```

11. find()

```
import pandas as pd
s = pd.Series(['Tom ', ' William Rick', 'John',\
               'Alber@t'])
print(s.str.find('e'))
```

Output:

```
0   -1
1   -1
2   -1
```

```
3    3
dtype: int64
```

12. swapcase()

```
import pandas as pd
s = pd.Series(['Tom', 'William Rick', 'John',\
               'Alber@t'])
print(s.str.swapcase())
```

Output:

```
0             tOM
1    wILLIAM rICK
2            jOHN
3         aLBER@T
dtype: object
```

13. islower()

```
import pandas as pd
s = pd.Series(['Tom', 'William Rick', 'John',\
               'Alber@t'])
print(s.str.islower())
```

Output:

```
0    False
1    False
```

```
2    False

3    False

dtype: bool
```

14. isupper()

```
import pandas as pd
s = pd.Series(['Tom', 'William Rick', 'John',\
               'Alber@t'])
print(s.str.isupper())
```

Output:

```
0    False

1    False

2    False

3    False

dtype: bool
```

Row and Column filtering in pandas

Select required variables/columns from Dataframe or Series for analysis. Pandas offer different methods to filter the data based on values or conditions.

Selecting Column A

```
import pandas as pd
import numpy as np
df = pd.DataFrame(np.random.randn(8, 4),\
```

```
                    columns = ['A', 'B', 'C', 'D'])
print(df['A'])
```

Output:

```
0   -1.182770
1    1.551252
2   -0.374627
3    1.162744
4   -0.340436
5    0.436685
6    1.605121
7   -1.507549
Name: A, dtype: float64
```

Note – We can pass a list of values to [ ] to select those columns.

Selecting Column A and B

```
import pandas as pd
import numpy as np
df = pd.DataFrame(np.random.randn(8, 4),\
                    columns = ['A', 'B', 'C', 'D'])
print(df)
print (df[['A','B']])
```

Output:

```
A         B         C         D
0  1.429404  0.004506 -0.233915 -1.521801
1 -0.875495 -0.459941  0.110985 -0.660796
2  0.198817  1.367113 -0.158370 -1.745277
3 -0.916441  1.333897 -0.666410 -2.214911
4  0.250153  1.972655 -0.478166 -1.562573
5  0.969860 -0.420313 -1.516736 -1.433613
6 -0.271332 -1.025844  1.303945  0.695108
7  1.608772  1.433876 -0.780303 -0.235205
          A         B
0  1.429404  0.004506
1 -0.875495 -0.459941
2  0.198817  1.367113
3 -0.916441  1.333897
4  0.250153  1.972655
5  0.969860 -0.420313
6 -0.271332 -1.025844
7  1.608772  1.433876
```

Row filter using indexing

```
import pandas as pd
import numpy as np
df = pd.DataFrame(np.random.randn(8, 4),\
                  columns = ['A', 'B', 'C', 'D'])
print(df[2:3])
```

output:

```
     A       B       C       D
2 -0.01857  1.089982  1.75753  0.848581
```

Attribute Access (Column)

Columns can be selected using the attribute operator '.'

```
import pandas as pd
import numpy as np
df = pd.DataFrame(np.random.randn(8, 4),\
                  columns = ['A', 'B', 'C', 'D'])
print(df.A)
```

output:

```
0   -0.754719
1   -0.353825
2    0.275724
```

```
3   -0.625538
4    2.238337
5    2.623854
6   -0.503629
7   -0.261538
Name: A, dtype: float64
```